TECHNICAL WHITEPAPER

Gima

Local-First AI Workspace

Memory, research, artifact creation, media workflows, provider routing, image/video generation, and review-gated self-improvement.

Inspectable | Local-first | Source-backed | Human-approved | Provider-ready

Version 2.6 | 4 July 2026 | Prepared by Gimhan Gunarathne

Portfolio, LinkedIn, GitHub, and technical partner presentation package

Author	Project	Version	Date
Gimhan Gunarathne	Gima / Human AI Local	2.6	4 July 2026

Whitepaper thesis

Gima is positioned as an AI operations cockpit: a visible workspace for memory, research, file generation, media planning, image/video generation, provider routing, and safe self-improvement.

Repository: https://github.com/data-G/Gima

Contents

This whitepaper is designed for a portfolio reader, GitHub reviewer, LinkedIn audience, or early technical partner. It separates product proof, architecture, capability status, and governance.

16+	Local	Cloud	Review
Capability tracks	Memory and artifacts	Optional OpenAI/OpenRouter	Human approval gates

Executive Snapshot

Gima turns a local machine into an inspectable AI workspace. Instead of hiding work inside a chat transcript, it routes requests through memory, browsing, deterministic artifact tools, optional cloud models, image/video generation backends, and review-gated improvement steps.

Chat	Hybrid	Files	Safe
Text, voice, and saved turns	Local plus optional providers	PDF, CSV, XLSX, PNG, MP4	Review before risky actions

Principle	What It Means
Local-first memory	Conversations, generated files, reviews, and logs are stored under the local Gima workspace.
Source-backed facts	Current information routes through browsing or APIs instead of stale memory guesses.
Real artifacts	Reports, source registers, screenshots, media plans, and exports are written to visible folders.
Conversational status	Gima can chat, save conversation history, wake on a spoken name, and speak responses when the local voice backend is available.
Human-controlled improvement	Code sync, public posting, financial actions, and self-updates require approval.

Professional thesis: useful personal AI should be auditable, multimodal, file-producing, provider-aware, and safe enough to improve continuously without becoming opaque.

Professional Capability Tracks

Gima is presented as a practical AI workspace with professional capabilities across engineering, research, privacy, and product delivery. These are capability directions backed by working modules, tests, docs, and roadmap items, not claims of full autonomous replacement.

Track	What Gima Supports	Proof Direction
Gima AI Engineer	Model routing, local-model fallback, teacher-model gateways, agent workflows, testing, evaluation, and safe self-improvement.	Provider fallback tests, upgrade reports, capability dashboard, review-gated code changes
OSINT Research Architect	Source-backed public research, authorized research gates, citation registers, contradiction notes, research dossiers, and exportable evidence tables.	Source registers, cited reports, authorization gate, uncertainty flags, public-source exports
Privacy Engineer	Local-first memory, masked secrets, permission checks, cloud-use gating, provenance manifests, protected paths, and review queues.	CLOUD_ALLOWED gate, masked API keys, protected downloads, local memory paths
Full-Stack AI Builder	Python backend, browser UI, local files, artifact generation, API integrations, GitHub/deployment workflows, and user-facing product documentation.	Working web app, generated files, tested routes, GitHub sync docs, deployment guides

Positioning rule

Use these tracks for portfolio, sponsor, and LinkedIn communication, but keep language evidence-based: Gima assists, drafts, routes, tests, and documents. Public, financial, identity, and cloud actions remain user-approved.

Problem and Product Vision

Most AI assistants are powerful but opaque: users cannot always see which model answered, whether a current fact was checked, where files were saved, or what would happen if the system modified itself. Gima addresses this by treating AI as a workspace rather than a single chat box.

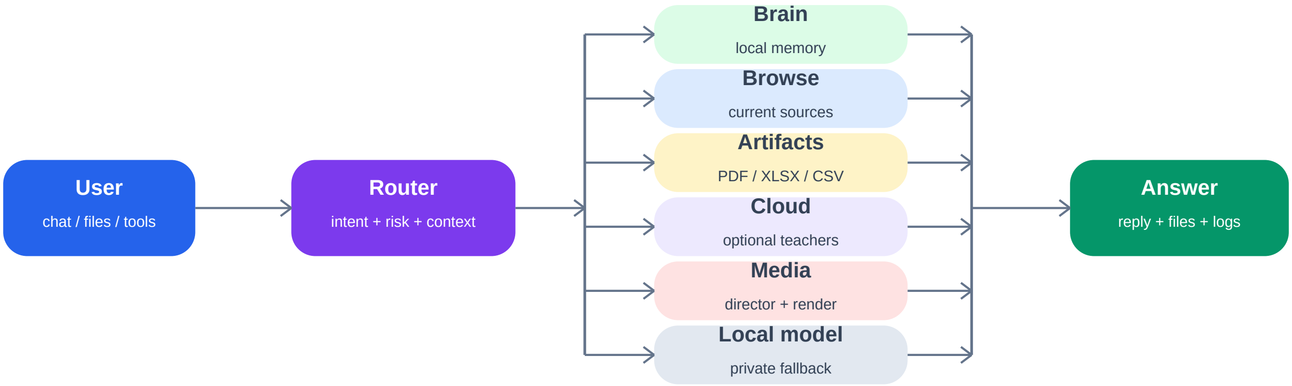
Need	Gima Response
Trust	Show sources, files, provider status, and memory paths.
Speed	Use small local routes for quick fallback and stronger providers when needed.
Professional output	Generate reports, costings, screenshots, PDFs, and structured exports.
Conversation	Support text chat, wake-word activation, spoken responses, and local voice conversation logs.
Media creation	Plan scenes, camera angles, emotions, audio timing, and backend render steps.
Growth	Prepare legal earning assets while keeping public, financial, and identity actions user-approved.

Target experience: the user asks for a business report, web research, media plan, code change, or GitHub sync. Gima chooses the correct route, creates visible outputs, logs the result, and asks for approval when risk increases.

System Architecture

Gima is organized around a router. Each request is classified by intent, risk, and context, then sent to memory, browsing, artifact generation, cloud models, local fallback, or media workflows.

Figure 1 - Gima Routing Architecture



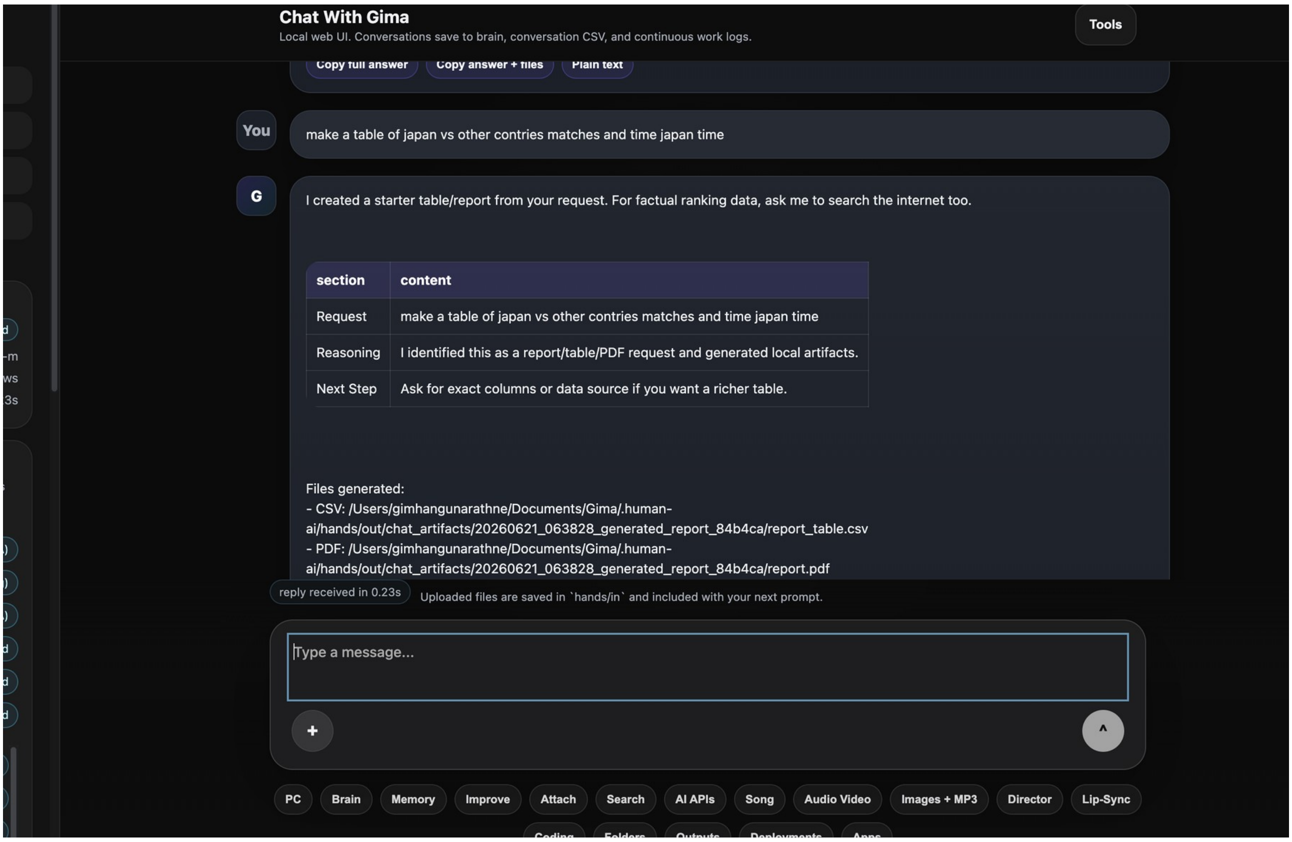
Routing makes Gima a workspace: every answer can carry memory, sources, generated files, and review state.

Layer	Responsibility	Implementation
Interface	Chat, uploads, dashboards, tool buttons	human_ai/web_ui.py
Memory	Records, reviews, brain index, source files	.human-ai, memory.py, brain_index.py
Routing	Select brain, browse, artifact, media, cloud, or local path	agent.py, artifacts.py, services.py
Providers	Optional OpenAI, Gemini, Anthropic, OpenRouter routes	services.py, config
Artifacts	PDF, CSV, Markdown, images, video manifests	hands/out, scripts
Governance	Secrets, approval gates, quotas, sync checks	secrets.py, quota.py, self_update.py

Product Screenshots

The screenshots below are included as product evidence. They show the current Gima workspace UI, generated-output behavior, and early multimodal controls.

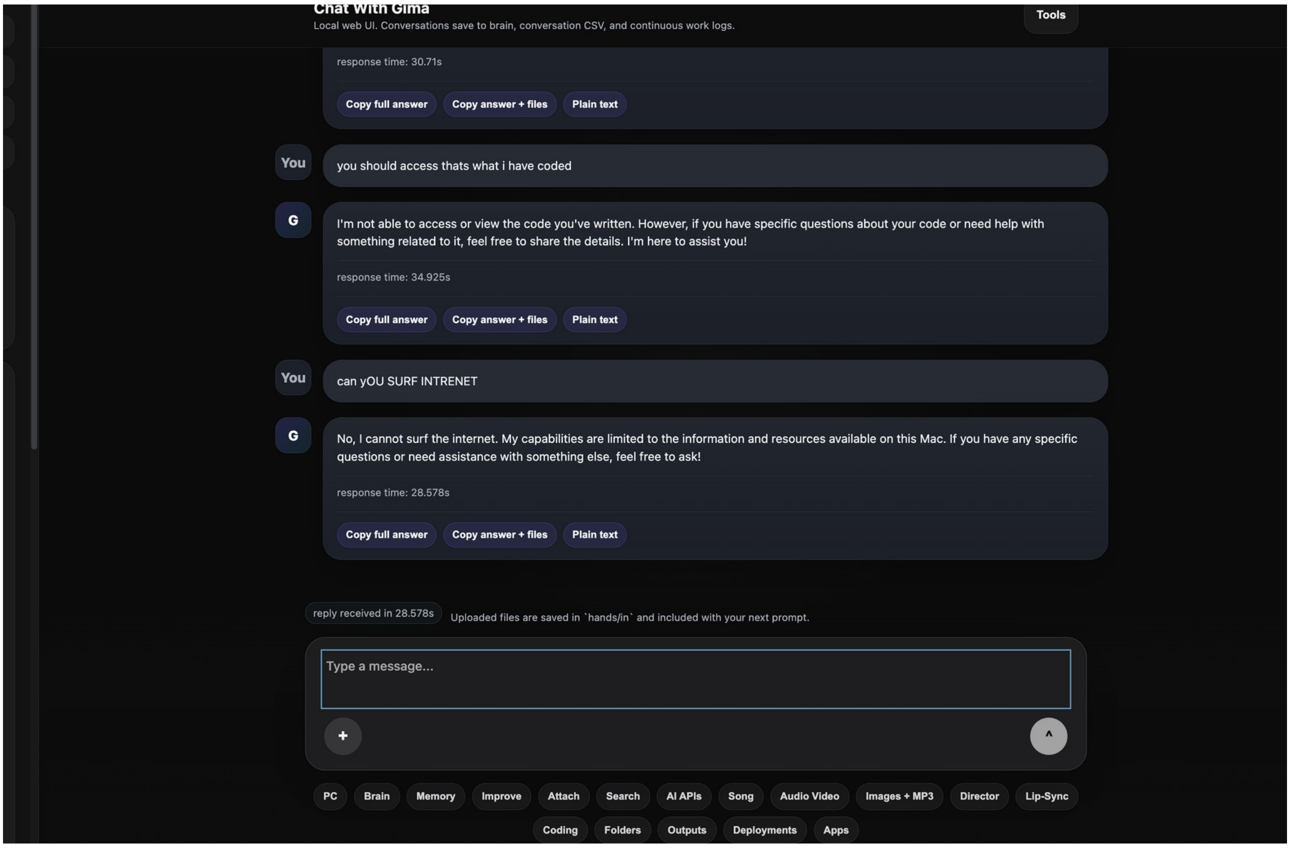
Artifact and Report Workflow



Gima recognizes a table/report request and returns saved CSV/PDF style outputs instead of only a chat answer.

Product Screenshots

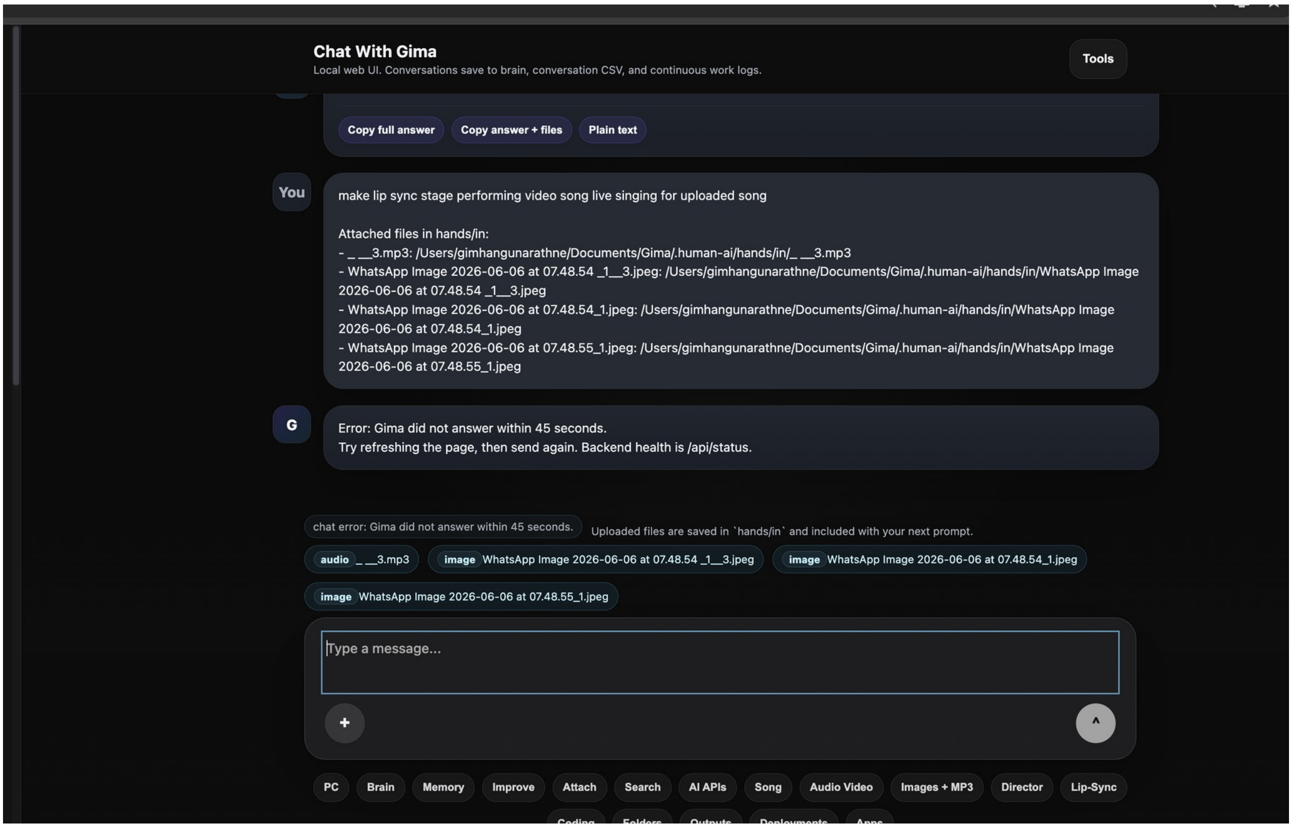
Chat Workspace and Tool Controls



The interface exposes copy/export controls, feature buttons, memory/provider state, and workspace navigation.

Product Screenshots

Media and Lip-Sync Workflow Controls



The product direction includes uploaded image/audio workflows, director planning, and lip-sync/rendering routes.

Capability Model

Capability	Current / Planned Behavior	Professional Output
Research	Browse or import public sources, summarize, and save source metadata.	Cited notes, CSV registers, PDF briefings
Authorized security audit	Ask ownership, permission, scope, allowed/prohibited actions, and private-report preference before security or reverse-engineering-style work.	Private responsible report, safe public-only fallback
Costing and tables	Build assumption-led estimates and export visible files.	Excel-style workbooks, JPG previews, PDFs
Conversational AI	Hold text conversations, save history, wake on spoken Gima, and speak replies where voice tools are available.	Conversation CSV, voice turns, wake events
Provider routing	Use local fallback or optional cloud models based on task difficulty.	Provider-aware answer logs
Image generation	Generate ChatGPT/OpenAI images from prompts with consent and local provenance manifests.	PNG output, prompt file, manifest
Veo video generation	Submit OpenRouter/Veo cloud video jobs, poll status, download MP4, and save usage/cost metadata.	MP4 output, job ID, manifest
Media director	Analyze prompt/audio, create scenes, camera moves, emotion map, and render plan.	Shot list, storyboard, timing manifest
GitHub sync	Check CLI auth, scan for obvious secrets, commit, push, and open draft PR.	Reviewable branch and PR
Agent workbench	Plan-act-observe loops with logs and human checkpoints.	Resumable task ledger

The strongest near-term product advantage is not claiming frontier autonomy. It is proving reliable workflow conversion: prompt to route, route to file, file to review, review to improvement.

Conversational Status

Gima is conversational first. The user should be able to type, speak, ask follow-up questions, and see whether Gima is listening, thinking, speaking, saving memory, or waiting.

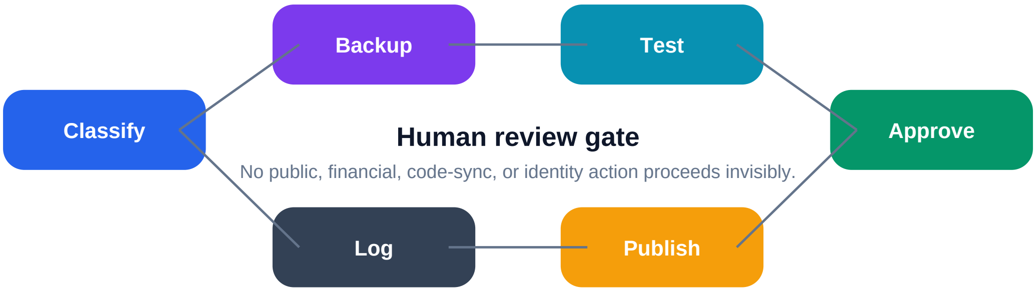
Area	Status	Implementation / Evidence
Text chat	Working	Web UI chat sends turns through Gima and saves conversations locally.
Conversation memory	Working	MemoryStore writes user and assistant turns into searchable conversation CSV files.
Wake word	Working / tested	WakeAssistant detects Gima and aliases while avoiding word-fragment false matches.
Speak replies	Working where backend is available	Voice().speak(...) is used in wake and assistant flows.
Direct voice conversation	Started	LocalAssistant.run_conversation(...) supports turn-taking, cleanup of filler transcripts, and stop phrases.
Multilingual speech normalization	Started	Unicode-preserving normalization supports mixed-language transcripts.
Realtime browser voice UI	Planned	Add microphone controls, live transcription, streaming speech, interruption, and visible status states.

Status State	Meaning
Listening	Microphone or wake-word flow is active with user permission.
Thinking	Gima is routing the request through memory, model, tool, browse, or artifact flow.
Speaking	A local or configured voice backend is reading the reply aloud.
Saving memory	The transcript and response are being written to local conversation history.
Waiting	Gima is idle and ready for the next typed or spoken turn.

Voice must remain user-controlled: no silent recording, clear microphone permission, optional speech output, and an immediate stop phrase such as end game.

Safety and Governance

Gima uses application controls in addition to prompts. This keeps learning, code changes, publishing, and money-related workflows visible and reviewable.



Risk	Control
API key exposure	Masked UI, local secrets files, Git ignore rules, and pre-sync secret scans.
Fake current facts	Route current information through browsing or APIs.
Unsafe automation	Require approval for GitHub sync, public posts, financial actions, and purchases.
Media misuse	Require consent gates and provenance manifests for image/audio/video workflows.
Bad self-improvement	Use backup, patch, test, review, and rollback paths instead of silent live mutation.

Criticism and Defense Matrix

The whitepaper deliberately avoids claiming unchecked autonomy. Gima is strongest when positioned as a review-gated workspace that can produce visible outputs, tests, logs, and rollback paths.

Potential criticism	Why it matters	Defense built into Gima
Not fully autonomous	Gima should not claim complete unsupervised autonomy.	Frame autonomy as scoped, logged, reversible, and approval-gated for spending, posting, deployment, client contact, and live self-editing.
Evaluation still needed	Architecture claims require benchmark evidence.	Publish benchmark prompts, metrics, output samples, failure cases, and regression tests.
Local storage can still leak	Local-first does not automatically mean secure.	Use masked keys, secret scanning, permissioning, backups, recovery tests, and an encryption roadmap.
RAG can still be wrong	Source-backed answers can misread sources.	Add contradiction notes, citation validation, quote boundaries, uncertainty flags, and source freshness checks.
Artifact tools can fail	Files may generate with formatting, schema, formula, or rendering errors.	Use open-file checks, visual QA, schema checks, manifests, and repair loops.
Self-improvement can regress	Code changes may break the system.	Require backup, isolated copy, tests, diff review, rollback path, and release notes before sync.

Positioning rule

Gima should be presented as practical, inspectable, and improving - not as a magic autonomous agent. Its credibility comes from outputs, controls, and tests.

Roadmap

Reliability P0	Stable startup, health checks, provider status, clean restart behavior.
Conversation P0	Visible listening/thinking/speaking/saving states, voice controls, and conversation memory quality checks.
Research P0	Cited research dossiers, source registers, and contradiction notes.
Artifacts P0	Professional Excel/PDF/JPG workflows with assumptions and manifests.
Provider Layer P1	OpenRouter model picker, MiniMax tests, streaming, usage and cost logs.
Media P1	True video backend adapters, scene planner, audio analysis, lip-sync evaluation.
Agents P2	Resumable plan-act-observe workbench with progress UI and completion tests.
Public Release P2	License, GitHub release, demo video, LinkedIn launch, whitepaper package.

Daily Improvement Loop

Track	Daily Evidence
Reliability	Status, startup, hidden errors, and smoke tests.
Knowledge	Source-backed learning and brain rebuild.
Artifact	One real output bundle.
Legal earning	A truthful portfolio, proposal, or LinkedIn asset.
Evaluation	Focused tests or live product checks.
Safe self-improvement	Backup, diff, tests, and approval.

LinkedIn-Ready Positioning

I am building Gima, a local-first AI workspace that combines private memory, web browsing, real artifact generation, optional cloud models, media planning, and safe self-improvement.

The idea is simple: a useful personal AI should not only chat. It should remember locally, browse when facts are current, create real files, show sources, protect secrets, and improve through tests and user approval.

Gima is still experimental, but it is becoming a practical AI workspace for research, reports, media planning, coding, GitHub sync, and daily improvement.

Conclusion

Gima's value is not one model. Its value is the system around the model: local memory, source-backed browsing, deterministic artifact routes, optional cloud intelligence, visible files, safety controls, tests, and user approval.

Reference	URL
Local-first software	https://www.inkandswitch.com/essay/local-first/
Retrieval-Augmented Generation	https://arxiv.org/abs/2005.11401
OWASP GenAI Security Project	https://genai.owasp.org/
NIST AI Risk Management Framework	https://www.nist.gov/itl/ai-risk-management-framework
OpenRouter Documentation	https://openrouter.ai/docs
Gima source repository	https://github.com/data-G/Gima